UNIVERSITÄT
LEIPZIG

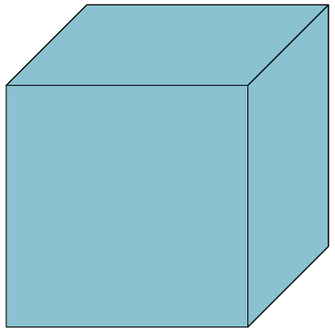
Java

Entwicklung verteilter Anwendungen

Leipzig 21.04.2022

Dr. Pascal Kovacs

AGENDA



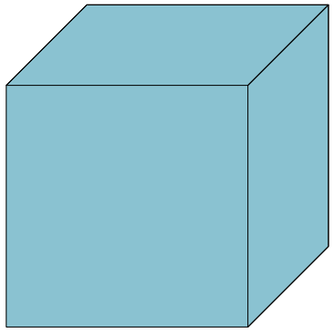
Java

ZIELE DER LETZTEN VERANSTALTUNG

- Java als Plattform

BEISPIELFRAGE

AGENDA



Java

ZIELE

- Java als Sprache
- Anweisungen, Operatoren
- Objektorientierung, Vererbung und Interfaces
- Exceptionhandling, Collections, Wrapperklassen

PROGRAMMIERSPRACHE JAVA

- Designziele
  - Objektorientiert
  - Einfach, vertraut (Notation ähnlich zu C++)
  - Nebenläufig
  - Dynamisch
  - Robust, sicher (eigentliches Ziel: *embedded*-Bereich)
  - Plattformunabhängig und portabel (Interpretiert)
  - Performanz
- Ermöglicht Entwicklung wiederverwendbarer Softwarekomponenten
- Kam als einzige Sprache neben ABAP auch in SAP zum Einsatz
  - Nicht mehr von SAP forciert und an Community abgegeben

Quelle: http://java.sun.com/docs/white/langenv/

(1) HELLO WORLD

Snippet 1 HelloWorld

- Klassen
- Startmethode *static void main(String args[])*
- Konsolenausgabe
- Ausdrücke
- Blöcke
- Pakete
  - Jedes Paket/Unterpaket entspricht auf Dateiebene einem Verzeichnis
- Kommentare

(2) VARIABLEN, ATTRIBUTE, METHODEN, OBJEKTE

Snippet 2 Circle

- Variablen Deklaration
- Attribute
  - Sichtbarkeiten
  - Zugriff Getter und Setter
  - This
- Methoden
- Objekte
  - Initialisierung
  - Methodenaufruf

(2) PRIMITIVE VS. REFERENZDATENTYPEN

- Primitive Datentypen (werden mit `0` bzw. `false` initialisiert)
  - `boolean` : 8 Bit (true | false)
  - `byte` : 8 Bit
  - `short` : 16 Bit
  - `char` : 16 Bit
  - `int` : 32 Bit
  - `long` : 64 Bit
  - `float` : 32 Bit
  - `double` : 64 Bit
  - In Java gibt es kein `unsigned`
- Referenzdatentypen (werden mit `null` initialisiert)
  - Objekte
  - Arrays

(2) METHODENDEKLARATION

- Sichtbarkeitmodifizierer
  - `public, protected, private, ohne(paketsichtbar)`
- `static`
  - Methode ist auf Klassenebene ohne ausführendes Objekt verfügbar
- `synchronized`
  - Unterschiedliche Threads können nur disjunkt zugreifen
- `final`
  - Methode kann nicht überschrieben werden
  - Aber Deklaration einer Methode mit selben Namen, jedoch unterschiedlicher Signatur möglich
- `abstract`
  - Abstrakte Methoden
  - Legen Aufrufstruktur, nicht aber die Implementierung fest

(2) BEZEICHNER

- Es sollten in der Regel englische Bezeichner verwendet werden
- Strenge Unterscheidung zwischen Groß-/Kleinschreibung
- Bezeichner dürfen nicht mit einer Ziffer beginnen
- Keine Sonderzeichen in Bezeichnern
- Keine Leerzeichen in Bezeichnern
- Keine Verwendung reservierter Schlüsselwörter
- Klassennamen mit großem Anfangsbuchstaben und CamelCase
- Pakete: Nur kleine Buchstaben und Trennung mehrerer Wörter mit Unterstrich, besser noch Unterpakete
- Variablen, Attribute und Methoden: Kleiner Anfang und CamelCase

(3) ARRAYS, SCHLEIFEN, BEDINGUNGEN, OPERATOREN

Snippet 3 Prime

- Arrays initialisieren und verwenden
  - `<Typ>[] <ArrayName> = new <Typ>[<Anzahl>];`
  - `Zugriff: <ArrayName>[<Index>]`
  - 0-basierter Index
- Schleifen
  - Do-While
  - While
  - For
- If-Bedingung
- Operatoren

(3) OPERATOREN

- Operatoren:
  - Typ-Cast-Operator(vorsichtig verwenden): **(**<*Typ*>**)**
  - Arithmetische Operatoren: **+**, **-**, *****, **/**, **%**, **++**, **--**
  - (String-)Konkatenationoperator: **+**
  - Bitoperatoren: **~**, **&**, **|**, **^**, **<<**, **>>**, **>>>**
  - Vergleichsoperatoren: **==**, **!=**, **<**, **>**, **<=**, **>=**, **instanceof**
  - Bedingungsoperator (ternärer Operator): **?:**
  - Logische Operatoren: **!**, **&&**, **&**, **||**, **|**, **^**
  - Zuweisungsoperatoren: **=**, **+=**, **-=**, ***=**, **/=**, **%=**, **&=**, **|=**, **^=**

- Bei Rechenoperation automatische Typanpassung zur Vermeidung von Überläufen
- Vorsicht bei Division von Ganzzahlen: **7 / 2 ergibt genau(!) 3**

(3) PRIORITÄTEN DER OPERATOREN

- 14 Stufen (hoch bis niedrig)
  - ++, --, Vorzeichen, ~, !, (typ)
  - *, /, %
  - +, -
  - <<, >>, >>>
  - <, <=, >, >=, instanceof
  - ==, !=
  - &
  - ^
  - |
  - &&
  - ||
  - ?:
  - =
  - *=, /=, %=, -=, <<=, >>=, >>>=, &=, ^=, |=
- Nutzen Sie explizite Klammerung für das Erreichen der Prioritäten!

(3) OPERATOR: INSTANCEOF

- Verwendung: <zuTestendesObjekt> `instanceof` <Referenztyp>
- Zur Anwendung auf Referenzdatentypen
- **Achtung:** Wenn das zu testende Objekt mit `null` belegt ist, resultiert der Test immer in falsch!
- Hinweis: Häufige Verwendung weist auf eine suboptimale Unterklassenbildung hin

```
public boolean equals(Object anderesObjekt) {
    if(anderesObjekt != null &&
            anderesObjekt instanceof Produkt) {
        Produkt anderesProdukt = (Produkt)anderesObjekt;
        return name.equals(anderesProdukt.getName());
    }
            return false;
}
```

(4) PARAMETERÜBERGABE

Snippet4 ValueReference

- *Call-by-Value*
  - Alle primitiven Datentypen
  - Änderungen wirken sich nur auf lokale Kopie des Parameters aus
- *Call-by-Reference*
  - Parameterübergabe als Verweis auf Variable (Referenz/Zeiger)
  - Alle Referenzdatentypen (inkl. Arrays)
  - Änderungen wirken sich immer auf übergebenen Parameter aus
  - Im Allgemeinen werden Parameter nur lesend genutzt
  - Änderungen an Objekten erfolgen mit direktem Methodenaufruf am Objekt selbst

(5) ABSTRAKTE KLASSEN, VERERBUNG, INTERFACES, KONSTRUKTOREN

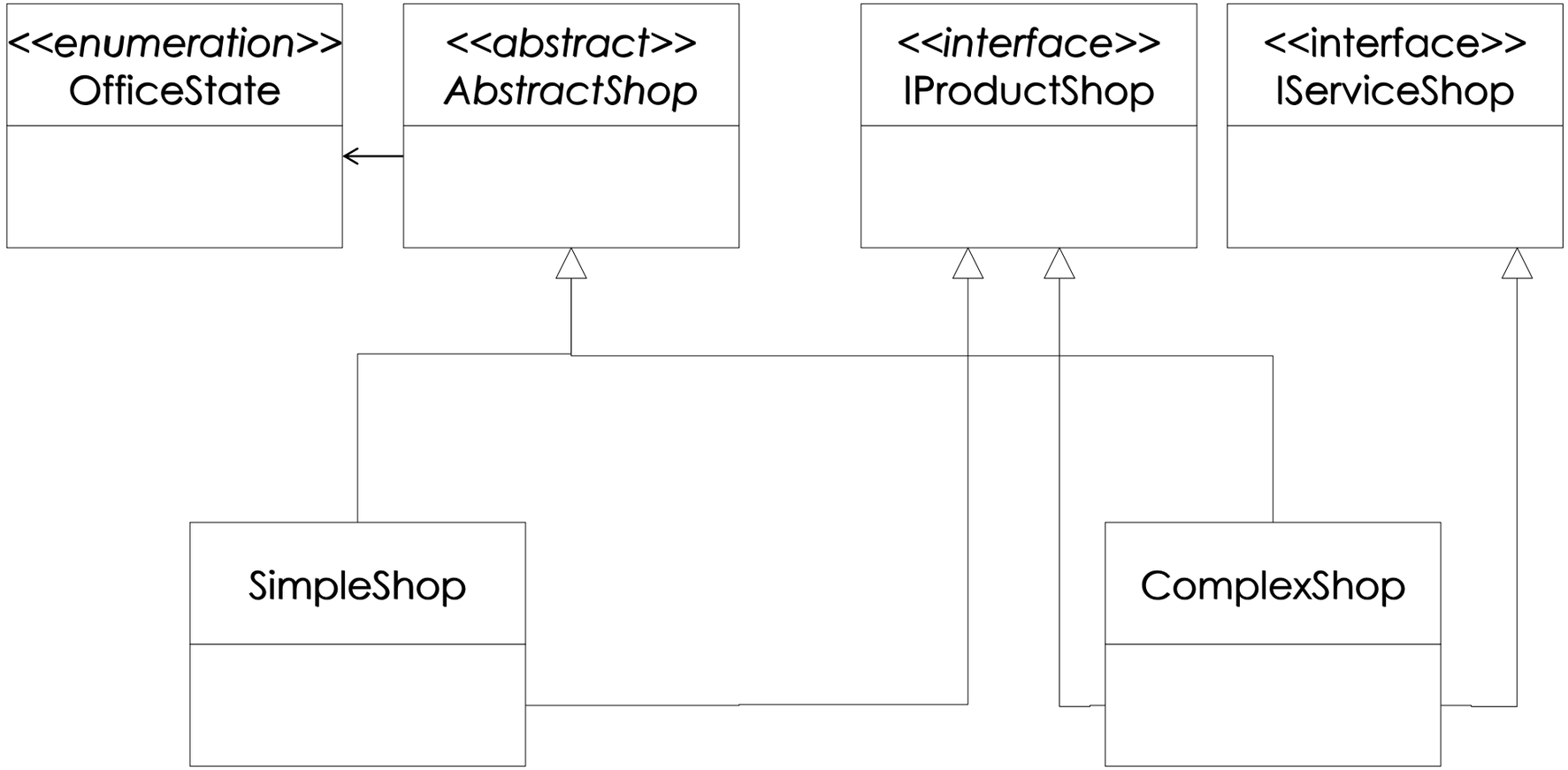
- Abstrakte Klasse
  - Enthält nicht implementierte Methodendeklarationen
  - Keine Instanziierung möglich
- Vererbung und Interfaces
  - In Java kann immer nur von einer Klasse geerbt werden
  - Dafür können mehrere Interfaces implementiert werden
- Konstruktoren
  - Automatische Erzeugung eines Standardkonstruktors ohne Parameter
  - Konstruktoren werden nicht vererbt
  - In Java gibt es keinen Destruktor!

(5) INTERFACES (SCHNITTSTELLEN)

- Klasse = Typ + Implementierung
- Interface = Typ (Verhaltensspezifikation)
  - Enthält nur abstrakte Methoden und Konstanten
    - Alle Methoden können erst in der realisierenden Klasse implementiert werden (außer abstrakten Klassen)
    - Alle deklarierten Operationen sind implizit `public abstract`
    - Alle definierten Attribute sind implizit `public static final` (Konstanten)
  - Keine Implementierung möglich
  - Keine Instanziierung möglich

(5) KLASSENDIAGRAMM SHOP

Snippet5 Klassenvarianten



(5) ENUM, SWITCH, KLASSE OBJECT

- Enum
  - Einzelwertaufzählungen
  - Alternative sind einzelne statische String Attribute
- Switch Anweisung
  - Löst mehrere If-Else-Anweisungen ab
  - Kein Performanzgewinn
- Klasse Object
  - Implizite Oberklasse aller Klassen
  - Stellt Methoden toString(), equals(`Object anotherObject`) und hashcode() zur Verfügung

(6) STRING, FOREACH-SCHLEIFE

Snippet6 Strings

- String als Klasse und nicht als primitiver Datentyp
- Verschiedene Methoden auf String ausführbar – Beispiele:
  - CharAt(int pos)
  - Substring(int pos, int length)
  - ToUpperCase() / toLowerCase()
  - matches(String regex)
  - split(String regex)
- Inhalt von Strings immer mit .equals() Methode prüfen
- Foreach-Schleife als Vereinfachung ohne Berücksichtigung der Größe eines Arrays

(7) STRÖME, EXCEPTIONS, IMPORT

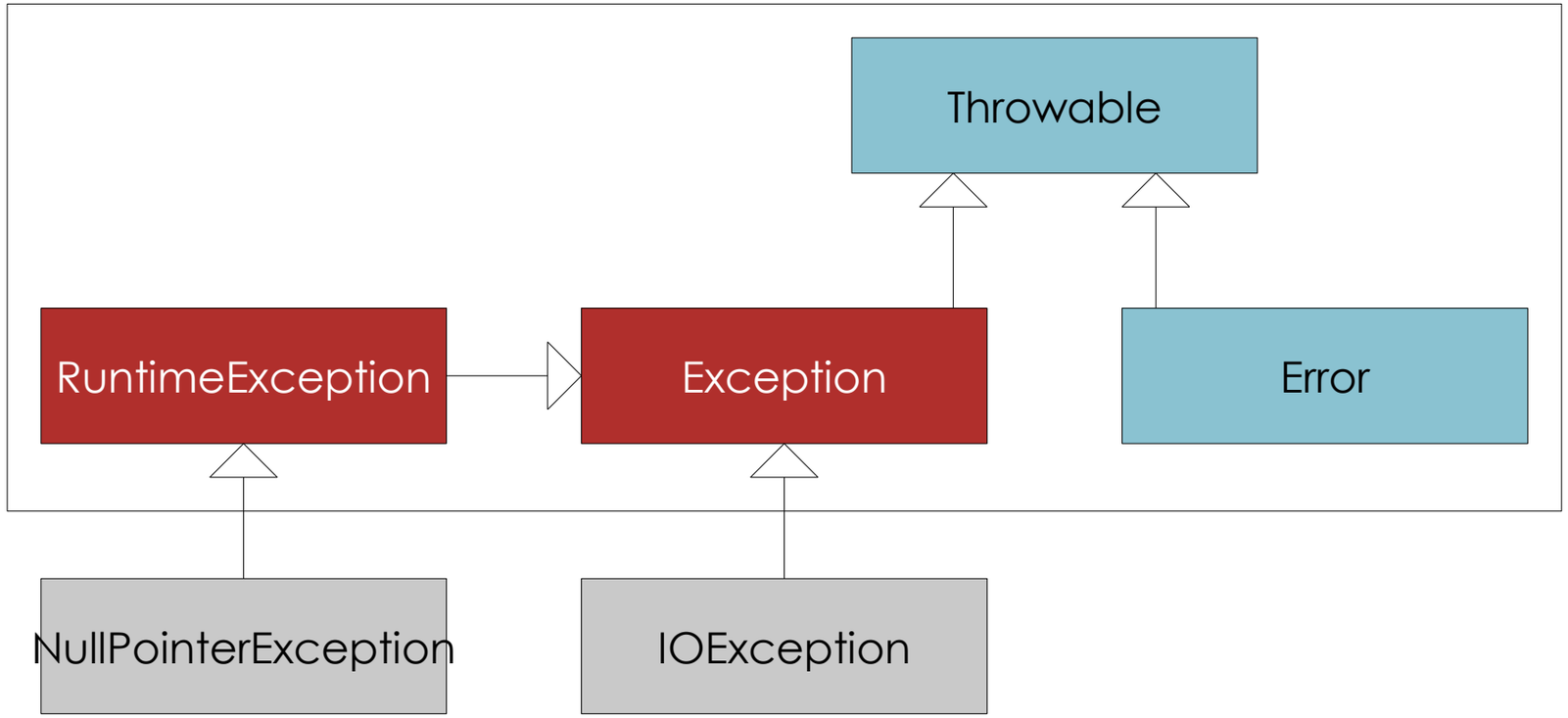
Snippet7 Streams

- Ströme werden verwendet um Daten einzulesen oder zu schreiben
  - Konsole
  - Dateien
  - Netzwerk
- Unterscheidung in Byte- und UniCode-Ströme
- Konzept von abstrakten Basisklassen mit Implementierungen für verschiedene Medien und Aufgaben
- Verkettung von Strömen für z.B. Filterungen/Pufferung möglich

(7) EXCEPTIONS

- Fehlerbehandlung durch Spezielle Objekte: Exceptions
  - Checked Exceptions müssen behandelt werden
  - Unchecked Exceptions können behandelt werden
- Exception-Klassen sind in Hierarchie organisiert → Spezialisierung
- Behandlung in *try catch* Blöcken durch „fangen“ der Exception
  - Mehrere *catch* Blöcke möglich
  - Immer von spezieller Exception zur allgemeineren Exception anordnen
  - Abschließende Arbeiten im *finally* Block durchführen
- Behandlung an aufrufende Methode mittels *throws* übergeben
- Es können auch eigene Exceptions definiert werden

(7) EXCEPTIONHIERARCHIE



(7) STRÖME, EXCEPTIONS, IMPORT

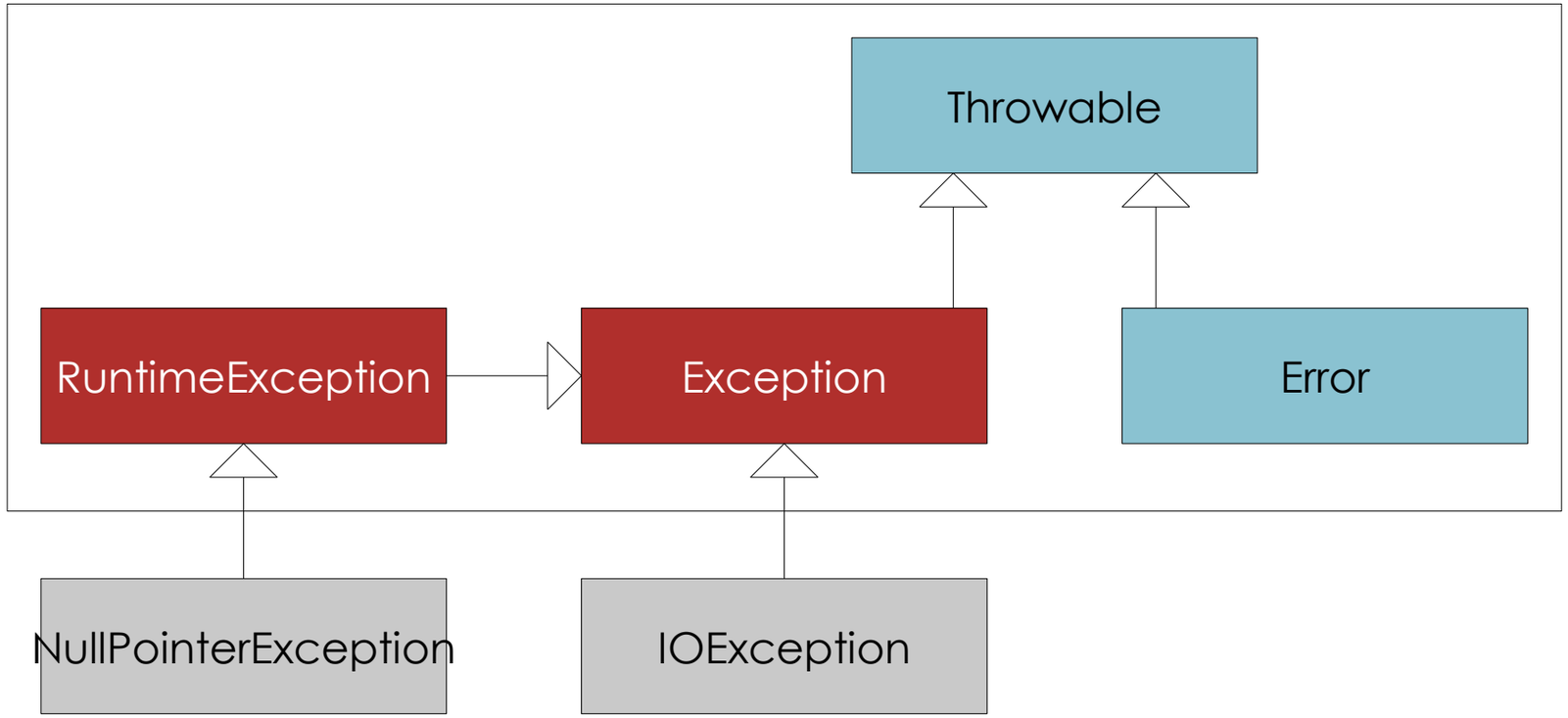
Snippet7 Streams

- Ströme werden verwendet um Daten einzulesen oder zu schreiben
  - Konsole
  - Dateien
  - Netzwerk
- Unterscheidung in Byte- und UniCode-Ströme
- Konzept von abstrakten Basisklassen mit Implementierungen für verschiedene Medien und Aufgaben
- Verkettung von Strömen für z.B. Filterungen/Pufferung möglich

(7) EXCEPTIONS

- Fehlerbehandlung durch Spezielle Objekte: Exceptions
  - Checked Exceptions müssen behandelt werden
  - Unchecked Exceptions können behandelt werden
- Exception-Klassen sind in Hierarchie organisiert → Spezialisierung
- Behandlung in *try catch* Blöcken durch „fangen“ der Exception
  - Mehrere *catch* Blöcke möglich
  - Immer von spezieller Exception zur allgemeineren Exception anordnen
  - Abschließende Arbeiten im *finally* Block durchführen
- Behandlung an aufrufende Methode mittels *throws* übergeben
- Es können auch eigene Exceptions definiert werden

(7) EXCEPTIONHIERARCHIE



(8) COLLECTIONS, WRAPPERKLASSEN

Snippet 8 Collections

- Arrays müssen mit fester Größe initialisiert werden
- Sind somit nicht dynamisch erweiterbar/reduzierbar
- Paket java.util stellt verschiedene Klassen zur dynamischen Verwaltung von Sammlungen(Collections) bereit
- Zusätzlich nützliche Methoden wie die Suche nach Elementen
- Konzept beruht auf Interfaces mit verschiedenen Implementierungen
- 3 Interfaces für
  - *List* für Liste mit geordneten Elemente
  - *Set* für ungeordnete Menge von Elementen, ohne Dubletten
  - *Map* für eine Sammlung von Schlüssel-Wert-Paaren

(8) COLLECTIONS IMPLEMENTIERUNGEN

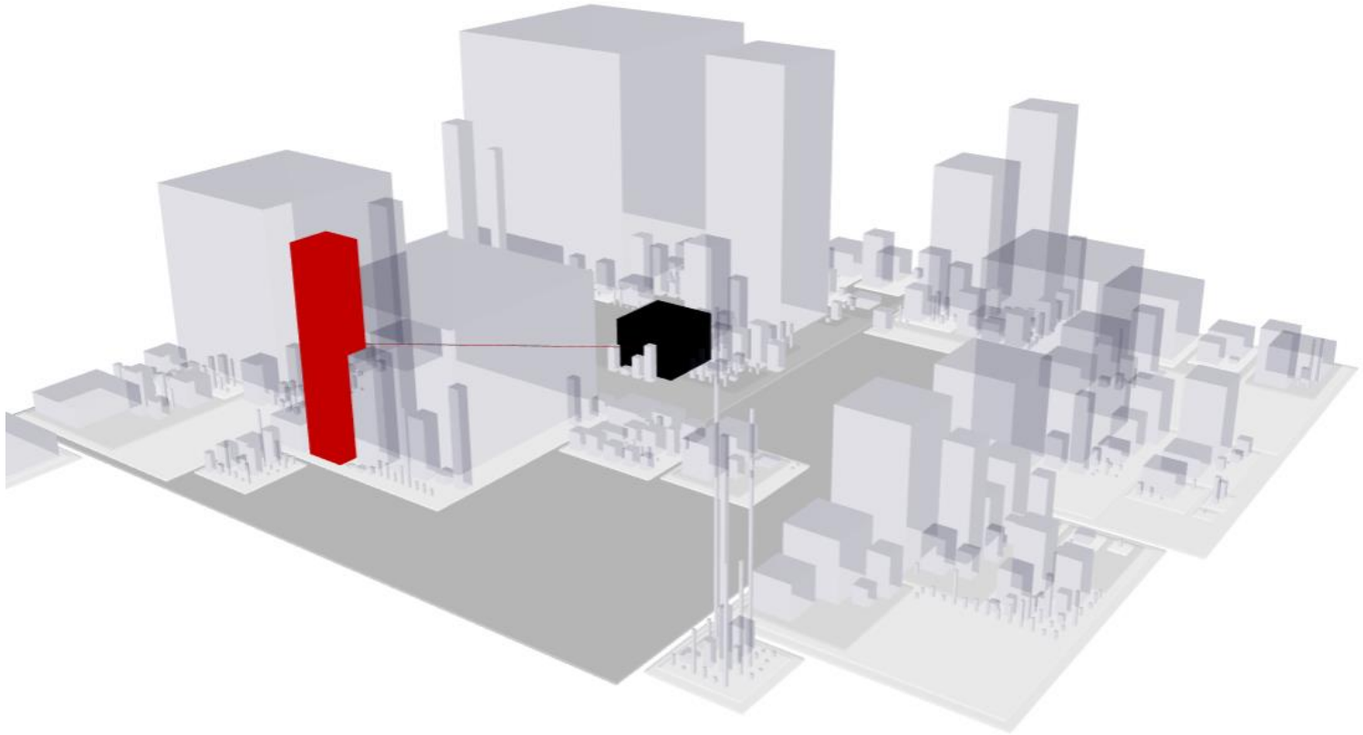
- Implementierungen von *List (Auswahl)*
  - *ArrayList* - schneller Index-Zugrif, langsames Einfügen(außer am Ende)
  - *Vector* – wie *ArrayList*, aber thread-sicher
- Implementierungen von *Set (Auswahl)*
  - *HashSet* – interne Hash-Tabelle, keine Reihenfolgegarantie bei Iteration
  - *TreeSet* – interne Baumstruktur, somit Sortierung möglich
- Implementierungen von *Map (Auswahl)*
  - *HashMap – Mapping mittels Hash-Werten*
  - *TreeMap – interne Baumstruktur, somit Sortierung möglich*

(8) WRAPPERKLASSEN

- Collections verwalten standardmäßig nur Objekte vom Typ Object
- Kann zu Fehlern führen, da Objekte von verschiedenen Typen in der gleichen Collection abgelegt werden können → TypeCast
- Seit Einführung von Generics sollten Collections daher mit einem bestimmten Typ parametrisiert werden
- Was passiert mit Primitiven Typen wie int oder float?
- Für diese wurden spezielle Wrapperklassen eingeführt, z.B. Integer
- Kapseln einen Wert des Typen in einem Objekt
- Objekte können aber weiterhin wie normaler primitiver Typ verwendet werden
- Entsprechende Aufrufe werden automatisch angepasst (Boxing)

UNIVERSITÄT
LEIPZIG

VIELEN DANK!



Pascal Kovacs

Institut für Wirtschaftsinformatik

pkovacs@uni-leipzig.de

https://www.wifa.uni-leipzig.de/iwi/